# Midterm Exam II

CAS CS 320: Concepts of Programming Languages

March 31, 2026

Name:

BUID:

- ▷ You will have approximately 75 minutes to complete this exam. Make sure to read every question, some are easier than others.
- ▷ Do not remove any pages from the exam.
- ▷ Make very clear what your final solution for each problem is (e.g., by surrounding it in a box). We reserve the right to mark off points if we cannot tell what your final solution is.
- ▷ You must show your work on all problems unless otherwise specified. A solution without work will be considered incorrect (and will be investigated for potential academic misconduct).
- ▷ Unless stated otherwise, you should only need the rules provided **in the problem statement** for any derivation.
- ▷ We will not look at any work on the pages marked "*This page is intentionally left blank.*" You should use these pages for scratch work.

| Problem # | Points |
|---|---|
| 1A | 5 |
| 1B | 2 |
| 2A | 5 |
| 2B | 5 |
| 3A | 5 |
| 3B | 5 |
| 4 | 5 |
| **Total** | 30 |

*This page is intentionally left blank.*

# 1 Higher-Order Programming

A. (5 points) Implement the function

```
val fold_left2 : ('acc -> 'a -> 'b -> 'acc) -> 'acc -> 'a list -> 'b list -> 'acc
```

so that `fold_left2 op base l r` is the result of left-associatively folding `op` over the lists `l` and `r` simultaneously, e.g.,

```
fold_left2 op base [a1;a2;a3;a4] [b1;b2;b3;b4]
```

is equivalent to

```
op (op (op (op base a1 b1) a2 b2) a3 b3) a4 b4
```

The behavior of the function is undefined if `l` and `r` are not the same length. **Your solution must be tail-recursive.** You *cannot* use any functions from the standard library.

B. (2 points) Recall that, given an $(m \times n)$ matrix $A$ and an $n$-dimensional vector $\mathbf{v}$, the matrix-vector product $A\mathbf{v}$ is then $m$-dimensional vector whose $i^{\text{th}}$ entry is $\sum_{j=1}^{n} A_{ij}\mathbf{v}_j$. In other words, the $i^{\text{th}}$ entry is the dot product of the $i$th row of $A$ and $\mathbf{v}$.

We can give an simple definition of matrix-vector multiplication using higher-order functions. We represent a vector as a list of floating-point numbers, and a matrix as a list of rows (each represented by lists of floating-point numbers).

```
type vec = float list
type mat = float list list

let op = assert false (* TODO *)
let base = assert false (* TODO *)
let mat_vec_mul (a : mat) (v : vec) : vec = List.map (List.fold_left2 op base v) a

let a = [[1. ; 2.] ; [3. ; 4.]]
let v = [5. ; 6.]
let _ = assert (mat_vec_mul a v = [(1. *. 5. +. 2. *. 6.); (3. *. 5. +. 4. *. 6.)])
```

Give definitions for `op` and `base` so that the above definition of `mat_vec_mul` is correct. Note that the behavior of this function is undefined if $A$ and $\mathbf{v}$ are not the same length (i.e., if $A$ has more or fewer rows than $\mathbf{v}$ has entries) or if $A$ is not a valid matrix. *Hint:* The only standard library you're allowed to use are `(+.)` and `( *. )` (i.e., floating-point addition and multiplication).

## 2 Formal Grammar

A. (5 points) Consider the following OCaml-like grammar (with start symbol <p>).

```
<p>  ::= <s> <p> | <s>
<s>  ::= let <v> = <e>
<e>  ::= let <v> = <e> in <e> | fun <v> -> <e> | <e2>
<e2> ::= <e2> <e3> | <e3>
<e3> ::= x | y | z | 0 | 1 | 2 | ( <e> )
<v>  ::= x | y | z
```

Demonstrate that the sentence

```
let x = let y = 0 in y
let z = (fun x -> x) 1
```

is recognized by the above grammar by providing a parse tree for it.

B. (5 points) Suppose we wanted to allow expressions at the top-level in OCaml. Then we could write:

```
let x = "foo"
let y = "bar"
print_endline x
print_endline y
```

This would require something like the following modification to the grammar from the first part (only two production rules for <p> have been added).

<p> ::= <s> <p> | <s> | <e> <p> | <e>
<s> ::= let <v> = <e>
<e> ::= let <v> = <e> in <e> | fun <v> -> <e> | <e2>
<e2> ::= <e2> <e3> | <e3>
<e3> ::= x | y | z | 0 | 1 | 2 | ( <e> )
<v> ::= x | y | z

Demonstrate that the above grammar is ambiguous by determining a sentence which has multiple parse trees. **Include both parse trees in your solution.**

## 3 Evaluation Strategies

Recall the big-step call-by-value (CBV) semantics for the $\lambda$-calculus:

$$\frac{}{\lambda x.e \Downarrow \lambda x.e}\ \textsc{fun} \qquad \frac{e_1 \Downarrow \lambda x.e \qquad e_2 \Downarrow v_2 \qquad e' = [v_2/x]e \qquad e' \Downarrow v}{e_1 e_2 \Downarrow v}\ \textsc{appCBV}$$

and the big-step call-by-name (CBN) semantics for the $\lambda$-calculus:

$$\frac{}{\lambda x.e \Downarrow \lambda x.e}\ \textsc{fun} \qquad \frac{e_1 \Downarrow \lambda x.e \qquad e' = [e_2/x]e \qquad e' \Downarrow v}{e_1 e_2 \Downarrow v}\ \textsc{appCBN}$$

A. (5 points) Give a closed[1] expression $e_0$ and a value $v_0$ such that $e_0 \Downarrow v_0$ is derivable using CBN semantics, but **not** derivable using CBV semantics. Your solution must include:

   (a) the CBN semantic derivation of $e_0 \Downarrow v_0$;

   (b) a one sentence description for why $e_0 \Downarrow v_0$ is not derivable using CBV semantics.

   *Hint.* Recall the expression $(\lambda x.xx)(\lambda x.xx)$.

[1] An expression is **closed** if it has no free variables.

B. (5 points) One notable feature of the semantic systems from the previous part is that they do not *evaluate under binders*. This is why we were able to ignore capture-avoidance when working with closed expressions. The *full* evaluation strategy (which does evaluate under binders, i.e., **note the difference in the FUN rule**) has the following semantic rules.

$$\frac{x \text{ is a variable}}{x \Downarrow x}\ \text{VAR} \qquad \frac{e \Downarrow v}{\lambda x.e \Downarrow \lambda x.v}\ \text{FUN} \qquad \frac{e_1 \Downarrow \lambda x.e \qquad e_2 \Downarrow v_2 \qquad e' = [v_2/x]e \qquad e' \Downarrow v}{e_1 e_2 \Downarrow v}\ \text{APP}$$

Determine a value $v$ up to $\alpha$-equivalence[2] such that

$$\lambda y.(\lambda x.\lambda y.x)y \Downarrow v$$

using the full evaluation strategy. **Also provide the derivation.** *Hint.* This will require $\alpha$-renaming. You may choose any fresh variable you'd like. The definition of capture-avoiding substitution is given on the following page.

[2]This means that your choice of $v$ can differ from ours in the naming of bound variables.

*(Problem 3B Continued)*

**Capture-Avoiding Substitution:**

$$[v/y]x = \begin{cases} v & x = y \\ x & \text{otherwise} \end{cases}$$

$$[v/y](\lambda x.e) = \begin{cases} \lambda x.e & x = y \\ \lambda z.[v/y][z/x]e & x \in FV(v), z \text{ is fresh} \\ \lambda x.[v/y]e & \text{otherwise} \end{cases}$$

$$[v/y](e_1 e_2) = ([v/y]e_1)([v/y]e_2)$$

# 4 Closures

(5 points) Recall the following environment-based semantic rules (which use only unnamed closures).

$$\frac{\text{n is an integer literal for } n}{\mathcal{E} \vdash \mathtt{n} \Downarrow n}\ \textsc{intLitE} \qquad \frac{\mathcal{E}(x) = v}{\mathcal{E} \vdash x \Downarrow v}\ \textsc{varE} \qquad \frac{}{\mathcal{E} \vdash \texttt{fun}\ x\ \texttt{->}\ e \Downarrow (\!|\, \mathcal{E}\ ,\ \texttt{fun}\ x\ \texttt{->}\ e \,|\!)}\ \textsc{funE}$$

$$\frac{\mathcal{E}_1 \vdash e_1 \Downarrow (\!|\, \mathcal{E}_2\ ,\ \texttt{fun}\ x\ \texttt{->}\ e \,|\!) \qquad \mathcal{E}_1 \vdash e_2 \Downarrow v_2 \qquad \mathcal{E}_3 = \mathcal{E}_2[x \mapsto v_2] \qquad \mathcal{E}_3 \vdash e \Downarrow v}{\mathcal{E}_1 \vdash e_1 e_2 \Downarrow v}\ \textsc{appE}$$

$$\frac{\mathcal{E}_1 \vdash e_1 \Downarrow v_1 \qquad \mathcal{E}_2 = \mathcal{E}_1[x \mapsto v_1] \qquad \mathcal{E}_2 \vdash e_2 \Downarrow v_2}{\mathcal{E}_1 \vdash \texttt{let}\ x\ \texttt{=}\ e_1\ \texttt{in}\ e_2 \Downarrow v_2}\ \textsc{letE}$$

Determine the (closure) value of the following expression. You do not need to provide a derivation, but you must show your work or include a short description indicating how you got your solution.

```
(fun f -> fun x -> f x) (let z = 1 in fun y -> y + z)
```

*This page is intentionally left blank.*

*This page is intentionally left blank.*